



GraphCastをgooglecolabで使ってみた(1/2)

2024/05/27に公開

天気予報

GraphCast

Tech

GraphCastとは

GraphCastとはGraph neural network(グラフニューラルネットワーク)を用いてforecast(天気予報)をするものです。従来の数値気象予測(NWP)に対して危害学習(ML)ベースの新しい手法です。

世界的な中期天気予報の精度がこの技術により向上しました。GraphCastは、再解析データから直接学習し、10日先までの気象変数を予測します。その予測精度は、現在最も精度の高い運用システム(ECMWFのHRES)を多くの評価対象で上回ります。

実際にGraphCastに触ってみる(モデルの作成まで)

以下のリンクからGoogleが出している公式のcolabファイルに触ることができます。

実装のための準備

graphcastに必要なコードをインストール

GraphCastのコードをgithubから持ってきます。

Workaround for cartopy crashes

このコードは、Google Colab環境でCartopyライブラリを使用する際に発生する可能性のあるクラッシュ問題を回避するためのものです。具体的には、Shapelyライブラリの特定のインストール方法を変更することで、互換性の問題を解決しようとしています。

背景

- Google Colabのデフォルト環境では、Shapelyライブラリがバイナリ形式でインストールされており、これがCartopyとの互換性問題を引き起こすことがあります。
- ソースからインストールすることで、ライブラリ間の依存関係やバージョンの不整合を避け、安定した動作を確保します。

import

必要なライブラリーをインポート

Authenticate with Google Cloud Storage

このコードは、Google Cloud Storage (GCS) に匿名クライアントとして接続し、特定のバケット("dm_graphcast")にアクセスするためのものです。

Plotting functions

気象データやその他の時系列データの可視化を行うための関数群です。特に、xarrayを使用してデータを操作し、matplotlibを使用してプロットを生成します。

データのロードとモデルの初期化

以下に、GoogleのDeepMindが提供する天気予報モデル「GraphCast」についての情報をまとめます。

モデルの取得方法

モデルのパラメータを取得する方法は2つあります。

1. ランダム(random):

- ランダムな予測を行いますが、モデルのアーキテクチャを変更できます。
- より高速に動作したり、デバイスに適合するかもしれません。

2. チェックポイント(checkpoint):

- センサブルな予測を行いますが、トレーニングされたモデルのアーキテクチャに限定されます。
- 勾配生成には多くのメモリが必要で、少なくとも25GBのRAM(TPUv4やA100)が必要です。

チェックポイントの種類

チェックポイントは以下の軸で異なります。

1. メッシュサイズ:

- 地球の内部グラフ表現のサイズです。小さいメッシュは高速ですが、出力の品質が低下します。
- メッシュサイズはモデルのパラメータ数に影響しません。

2. 解像度と圧力レベル:

- データに対応する解像度と圧力レベルです。低解像度で少ないレベルの方が高速に動作します。
- データの解像度はエンコーダー/デコーダーにのみ影響します。

3. 降水量の取り扱い:

- 全てのモデルが降水量を予測しますが、ERA5には降水量が含まれており、HRESには含まれていません。
- 「ERA5」モデルは降水量を入力として受け取り、ERA5データを期待します。
- 「ERA5-HRES」モデルは降水量を入力として受け取らず、HRES-fc0データを期待します。

提供される3つの事前学習済みモデル

1. GraphCast:

- 高解像度モデル(0.25度解像度、37圧力レベル)
- 1979年から2017年までのERA5データでトレーニングされています。

2. GraphCast_small:

- 低解像度の小型モデル(1度解像度、13圧力レベル、小さなメッシュ)
- 1979年から2015年までのERA5データでトレーニングされています。
- メモリと計算リソースが制限されている環境で有効です。

3. GraphCast_operational:

- 高解像度モデル(0.25度解像度、13圧力レベル)
- 1979年から2017年までのERA5データで事前学習され、2016年から2021年までのHRESデータでファインチューニングされています。
- HRESデータから初期化でき、降水量の入力を必要としません。

これらの情報を基に、適切なモデルを選択して使用することができます。

ランダムモデルの選択ウィジェット

ランダムモデルの設定を選択するためのスライダーとドロップダウンメニューが作成されます。

- メッシュサイズ (random_mesh_size): 内部グラフ表現のサイズを選択します。
- メッセージステップ (random_gnn_msg_steps): グラフニューラルネットワーク(GNN)のメッセージパス数を選択します。
- 潜在サイズ (random_latent_size): 潜在変数のサイズを選択します。
- 圧力レベル (random_levels): モデルの圧力レベルを選択します。

モデル選択

以上で設定したモデルを `Load the model` で設定し、ロードする。

exampleデータのロード

以下は、利用可能なデータセットの概要と、それらの特徴についての説明です。

利用可能なデータセット

データセットは以下の軸で異なります：

1. データソース:

- **fake**: 合成データ。
- **ERA5**: ECMWFの再解析データ。
- **HRES**: 高解像度予報システム。

2. 解像度:

- **0.25度**: 高解像度。
- **1度**: 中解像度。
- **6度**: 低解像度。

3. 圧力レベル:

- **13レベル**
- **37レベル**

4. タイムステップ:

- 含まれるタイムステップの数。

注意点

- **メモリ要件**: 高解像度データは、メモリ要件が高いため、タイムステップ数が少ない。
- **データ互換性**: データの解像度はロードされるモデルと一致している必要があります。
- **HRESの制限**: HRESデータは、0.25度の解像度で13の圧力レベルのみ利用可能。

データ変換

いくつかの変換がベースデータセットに対して行われています：

1. 降水量の積算:

- デフォルトの1時間積算ではなく、6時間積算の降水量。

2. HRESデータの特徴:

- 各タイムステップは、リードタイム0でのHRES予報に対応。これは、HRESの初期化を提供するもので、6時間積算の降水量は含まれません。

3. 太陽放射:

- データにはERA5の上層大気到達太陽放射 (toa_incident_solar_radiation) が含まれています。モデルは各1ステップ予測のために-6時間、0時間、+6時間の放射を使用します。
- データに放射が欠けている場合 (例: 運用環境では)、ERA5に類似した値を生成するカスタム実装が使用されます。

データセットのまとめ

データセットは多様な条件下での天気予報モデルの学習や評価に使用できます。選択するデータセットは、使用するモデルの仕様に一致する必要があります。特に解像度や圧力レベルに注意し、モデルの要求に合わせたデータセットを選択することが重要です。

利用可能なデータセットの取得

このコードは、利用可能なデータセットを取得し、現在のモデル構成およびタスク構成に基づいて適切なデータセットをフィルタリングするためのものです。以下は、コードの詳細な説明です。

利用可能なデータセットの取得

Google Cloud Storage (GCS) バケットから利用可能なデータセットのリストを取得します。

```
dataset_file_options = [  
    name for blob in gcs_bucket.list_blobs(prefix="dataset/")  
    if (name := blob.name.removeprefix("dataset/"))] # Drop empty s
```

データセットのフィルタリング

モデル構成およびタスク構成に基づいて、データセットがモデルに適合するかどうかをチェックします。

```
def data_valid_for_model(
    file_name: str, model_config: graphcast.ModelConfig, task_config: graphcast.TaskConfig
) -> bool:
    file_parts = parse_file_parts(file_name.removesuffix(".nc"))
    return (
        model_config.resolution in (0, float(file_parts["res"])) and
        len(task_config.pressure_levels) == int(file_parts["levels"]) and
        (
            ("total_precipitation_6hr" in task_config.input_variables and
             file_parts["source"] in ("era5", "fake")) or
            ("total_precipitation_6hr" not in task_config.input_variables and
             file_parts["source"] in ("hres", "fake"))
        )
    )
```

- **解像度のチェック:** モデルの解像度がデータセットの解像度と一致するか。
- **圧力レベルのチェック:** タスク構成の圧力レベル数がデータセットの圧力レベル数と一致するか。
- **データソースのチェック:**
 - 降水量 (`total_precipitation_6hr`) を必要とするモデルの場合、データソースがERA5またはfakeであること。
 - 降水量を必要としないモデルの場合、データソースがHRESまたはfakeであること。

ウィジェットの作成

フィルタリングされたデータセットのリストをドロップダウンメニューに表示します。

```
dataset_file = widgets.Dropdown(
    options=[
        ("", ".join([f'{k}: {v}' for k, v in parse_file_parts(option
        for option in dataset_file_options
        if data_valid_for_model(option, model_config, task_config)
    ],
    description="Dataset file:",
    layout={"width": "max-content"})
```

最終的なレイアウト

ドロップダウンメニューと説明ラベルを表示するためのレイアウトを作成します。

```
widgets.VBox([
    dataset_file,
    widgets.Label(value="Run the next cell to load the dataset. Return")
])
```

このコードを実行することで、現在のモデルに適合するデータセットをフィルタリングし、ユーザーが選択できるようになります。その後、次のセルでデータセットをロードすることができます。

データセットファイルの検証と読み込み (Load weather data)

選択されたデータセットファイルをGoogle Cloud Storageから読み込み、xarrayを使用して計算を行い、内容を表示するものです。以下は、コードの詳細な説明です。

選択されたデータセットファイルがモデルとタスクの構成に適しているかどうかを検証します。不適切な場合はエラーを発生させます。


```
if not data_valid_for_model(dataset_file.value, model_config, task_config):
    raise ValueError(
        "Invalid dataset file, rerun the cell above and choose a valid file"
```

Google Cloud Storageからのデータセットファイルの読み込み

Google Cloud Storageバケットから選択されたデータセットファイルを開き、xarrayを使用してデータセットを読み込み、計算を行います。

```
with gcs_bucket.blob(f"dataset/{dataset_file.value}").open("rb") as f:
    example_batch = xarray.load_dataset(f).compute()
```

時間次元の確認

データセットの時間次元が少なくとも3以上であることを確認します。2は入力用、1以上はターゲット用です。

```
assert example_batch.dims["time"] >= 3 # 2 for input, >=1 for target
```

データセットファイルの詳細情報の表示

データセットファイルの詳細情報（パーツ情報）を解析し、コンマ区切りで表示します。

```
print(", ".join([f"{k}: {v}" for k, v in parse_file_parts(dataset_file.value)]))
```

読み込まれたデータセットの表示

読み込まれたデータセット全体を表示します。

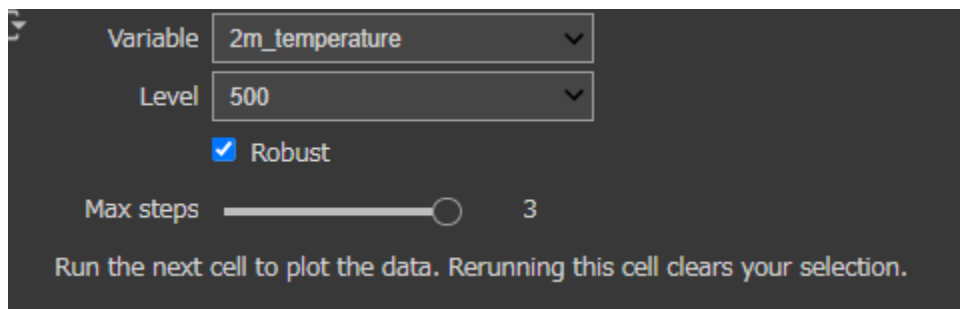
```
example_batch
```

このコードを実行することで、選択されたデータセットファイルを検証して読み込み、その内容を表示できます。

データのプロット

プロットするデータの選択 (Choose data to plot)

以下の図のように、プロットしたい要素とどの圧力帯でのプロットなのかを選択する。



Variable: 2m_temperature

Level: 500

☒ Robust

Max steps: 3

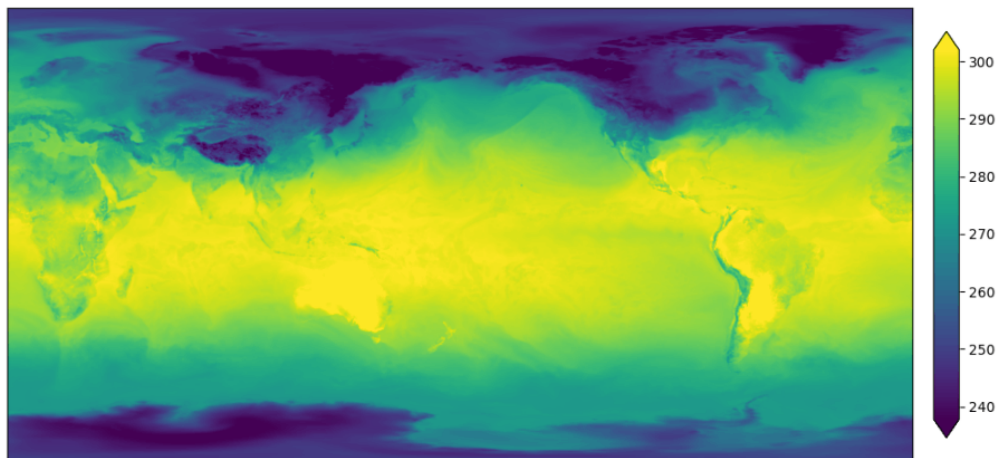
Run the next cell to plot the data. Rerunning this cell clears your selection.

データをプロット (Plot example data)

上で選択した内容を実際にプロットする。

プロットしたカラーマップの値が240~300となっているのは摂氏表記(°C)ではなくケルビン(K)標識であるからである。

0°C = 273 (K) であるので、240~300というのは、-33°C~27°Cを表している。



トレーニングデータと評価データ

トレーニングデータと評価データ

今までと同じように、トレーニングデータと評価データを選択する。

☐
Train steps

☐
Eval steps

Run the next cell to extract the data. Rerunning this cell clears your selection.

ここでの「step」は、トレーニングデータと評価データのタイムステップを指します。具体的には、時間軸に沿ったデータのスライスを意味します。タイムステップは、連続した時点におけるデータの観測値のセットです。

トレーニングステップ (Train steps) と評価ステップ (Eval steps)

- **トレーニングステップ (Train steps):**
 - モデルをトレーニングするために使用されるタイムステップの数を指定します。
 - 例えば、`train_steps` が 1 に設定されている場合、1つのタイムステップ分のデータがトレーニングに使用されます。
- **評価ステップ (Eval steps):**

- モデルを評価するために使用されるタイムステップの数を指定します。
- 例えば、`eval_steps` が `example_batch.sizes["time"]-2` に設定されている場合、トレーニングに使用されない残りの全てのタイムステップが評価に使用されます。
- **`train_steps` ウィジェット:**
 - デフォルト値: 1
 - 最小値: 1
 - 最大値: データの時間サイズ - 2
- **`eval_steps` ウィジェット:**
 - デフォルト値: データの時間サイズ - 2
 - 最小値: 1
 - 最大値: データの時間サイズ - 2

このコードは、ユーザーがトレーニングデータと評価データのためにどのタイムステップを使用するかを選択するためのインターフェースを提供します。タイムステップは、データの時間軸に沿った一連の観測値を指し、選択された数のステップがそれぞれトレーニングと評価に使用されます。

選択したデータをロードする (Extract training and eval data)

このコードは、選択されたタイムステップ数に基づいて、トレーニングデータと評価データを抽出するものです。具体的には、`data_utils.extract_inputs_targets_forcings` 関数を使用して、トレーニングおよび評価のための入力、目標、およびトレーニングデータを抽出します。

`data_utils.extract_inputs_targets_forcings`

この関数は、与えられたデータセットから指定されたリードタイム (予測時間) に基づいて入力、目標、およびトレーニングデータを抽出します。

- **パラメータ:**
 - `example_batch` : 元のデータセット
 - `target_lead_times` : 目標となるリードタイム (時間範囲)

- `task_config` : タスクの設定 (入力変数、目標変数、トレーニング変数、圧力レベルなど)

トレーニングデータの抽出

```
train_inputs, train_targets, train_forcings = data_utils.extract_inputs(
    example_batch, target_lead_times=slice("6h", f"{train_steps.value}h"),
    **dataclasses.asdict(task_config))
```

- `train_steps.value*6` : `train_steps` ウィジェットで選択されたステップ数に基づいてリードタイムを計算します (6時間単位)。
- `task_config` : タスクの設定を展開して渡します。

評価データの抽出

```
eval_inputs, eval_targets, eval_forcings = data_utils.extract_inputs(
    example_batch, target_lead_times=slice("6h", f"{eval_steps.value}h"),
    **dataclasses.asdict(task_config))
```

- `eval_steps.value*6` : `eval_steps` ウィジェットで選択されたステップ数に基づいてリードタイムを計算します (6時間単位)。

このコードを実行することで、選択されたタイムステップ数に基づいてトレーニングデータと評価データを抽出し、それぞれのデータセットの次元情報を確認することができます。

正規化データのロード(Load normalization data)

このコードは、Google Cloud Storage (GCS) バケットから正規化データを読み込むためのものです。これにより、データセットの標準化やスケーリングを行うための統計情報を取得します。以下は、コードの詳細な説明です。

Google Cloud Storageバケットから3つの統計ファイルを読み込み、`xarray` を使用してデータセットをロードします。

1. `diffs_stddev_by_level` の読み込み

各レベルごとの差分の標準偏差データを読み込みます。

```
with gcs_bucket.blob("stats/diffs_stddev_by_level.nc").open("rb") as f:
    diffs_stddev_by_level = xarray.load_dataset(f).compute()
```

2. `mean_by_level` の読み込み

各レベルごとの平均データを読み込みます。

```
with gcs_bucket.blob("stats/mean_by_level.nc").open("rb") as f:
    mean_by_level = xarray.load_dataset(f).compute()
```

3. `stddev_by_level` の読み込み

各レベルごとの標準偏差データを読み込みます。

```
with gcs_bucket.blob("stats/stddev_by_level.nc").open("rb") as f:
    stddev_by_level = xarray.load_dataset(f).compute()
```

正規化データの用途

これらのデータは、データセットの標準化やスケーリングに使用されます。標準化やスケーリングは、データの分布を均一にするための手法であり、機械学習モデルのトレーニングをより効果的に行うために重要です。

- **平均値 (mean):** データの中心位置を調整するために使用されます。
- **標準偏差 (stddev):** データの分散を調整するために使用されます。

- **差分の標準偏差 (diffs_stddev)**: 時間的な変動を考慮する場合に使用されることがあります。

これにより、モデルが異なるスケールや分布を持つデータに対しても安定して学習および予測できるようになります。

GraphCastモデルの構築とラッピング

このコードは、GraphCastモデルの構築、ラッピング、およびJAXを使用した関数のコンパイル（JIT化）を行います。これにより、モデルの学習と予測が効率的に実行されるようになります。以下に、各セクションの詳細な説明を提供します。

`construct_wrapped_graphcast` 関数

この関数は、GraphCastモデルを構築し、入力と出力のキャスト（型変換）および正規化を行うラッパーを追加します。

```
def construct_wrapped_graphcast(
    model_config: graphcast.ModelConfig,
    task_config: graphcast.TaskConfig):
    """Constructs and wraps the GraphCast Predictor."""
    # Deeper one-step predictor.
    predictor = graphcast.GraphCast(model_config, task_config)

    # Modify inputs/outputs to `graphcast.GraphCast` to handle conversion
    # from/to float32 to/from BFloat16.
    predictor = casting.BFloat16Cast(predictor)

    # Modify inputs/outputs to `casting.BFloat16Cast` so the casting to
    # BFloat16 happens after applying normalization to the inputs/targets.
    predictor = normalization.InputsAndResiduals(
        predictor,
        diffs_stddev_by_level=diffs_stddev_by_level,
        mean_by_level=mean_by_level,
        stddev_by_level=stddev_by_level)

    # Wraps everything so the one-step model can produce trajectories.
    predictor = autoregressive.Predictor(predictor, gradient_checkpointing)
    return predictor
```

1. GraphCastモデルの初期化:

- `graphcast.GraphCast` クラスを使って、指定されたモデル構成とタスク構成に基づいて予測器 (predictor) を作成します。

2. BFloat16キャストの追加:

- 入出力のデータ型をfloat32からBFloat16に変換するために、`casting.BFloat16Cast` を使用します。

3. 正規化ラッパーの追加:

- `normalization.InputsAndResiduals` クラスを使って、入力とターゲットのデータに対して正規化を適用します。

4. 自己回帰予測ラッパーの追加:

- `autoregressive.Predictor` クラスを使って、単一ステップのモデルが軌跡 (trajectory) を生成できるようにします。

関数のJIT化

`run_forward` 関数

この関数は、モデルの順伝播を実行します。

```
@hk.transform_with_state
def run_forward(model_config, task_config, inputs, targets_template,
               predictor = construct_wrapped_graphcast(model_config, task_config),
               return predictor(inputs, targets_template=targets_template, forcings=forcings))
```

`loss_fn` 関数

この関数は、モデルの損失関数を計算します。

```
@hk.transform_with_state
def loss_fn(model_config, task_config, inputs, targets, forcings):
    predictor = construct_wrapped_graphcast(model_config, task_config)
    loss, diagnostics = predictor.loss(inputs, targets, forcings)
    return xarray_tree.map_structure(
        lambda x: xarray_jax.unwrap_data(x.mean(), require_jax=True),
        (loss, diagnostics))
```

`grads_fn` 関数

この関数は、勾配を計算します。

```
def grads_fn(params, state, model_config, task_config, inputs, targets):
    def _aux(params, state, i, t, f):
        (loss, diagnostics), next_state = loss_fn.apply(
            params, state, jax.random.PRNGKey(0), model_config, task_config,
            i, t, f)
        return loss, (diagnostics, next_state)
    (loss, (diagnostics, next_state)), grads = jax.value_and_grad(
        _aux, has_aux=True)(params, state, inputs, targets, forcings)
    return loss, diagnostics, next_state, grads
```

ユーティリティ関数

with_configs 関数

この関数は、モデル構成とタスク構成を関数に渡すための部分適用を行います。

```
def with_configs(fn):
    return functools.partial(
        fn, model_config=model_config, task_config=task_config)
```

with_params 関数

この関数は、パラメータと状態を関数に渡すための部分適用を行います。

```
def with_params(fn):
    return functools.partial(fn, params=params, state=state)
```

drop_state 関数

この関数は、モデルの状態を無視して予測を行うためのラッパーを提供します。

```
def drop_state(fn):  
    return lambda **kw: fn(**kw)[0]
```

JITコンパイルの初期化

初期化されていない場合、ランダムな重みでモデルを初期化します。

```
init_jitted = jax.jit(with_configs(run_forward.init))  
  
if params is None:  
    params, state = init_jitted(  
        rng=jax.random.PRNGKey(0),  
        inputs=train_inputs,  
        targets_template=train_targets,  
        forcings=train_forcings)
```

JITコンパイルされた関数

```
loss_fn_jitted = drop_state(with_params(jax.jit(with_configs(loss_fn)  
grads_fn_jitted = with_params(jax.jit(with_configs(grads_fn)))  
run_forward_jitted = drop_state(with_params(jax.jit(with_configs(run
```

これらの関数を使用することで、GraphCastモデルの学習と予測が効率的に実行できるようになります。JITコンパイルにより、計算が高速化され、実行時のパフォーマンスが向上します。

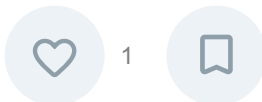
モデル準備までのまとめ

今回は、GraphCastを使うにあたって必要となるセッティングまでの解説を行いました。
具体的には

- 環境準備
 - ライブラリーのダウンロードとインポート
 - google cloud storage設定
 - プロットする関数の作成
- データとモデルの準備
 - RandomとCheckpointからデータセットを設定
 - データセット、解像度、プレッシャーレベルなどのモデル設定
 - データセットのプロット
 - トレーニングデータと評価データの設定
 - GraphCastモデルの構築、ラッピング、およびJAXを用いた関数のコンパイル

を行いました。

次回、セットしたデータセットとモデルを用いて学習を行います。



Discussion



ログインするとコメントできます

Login